



SOUTICK SAMANTA

REACT

FIRST EDITION

REACT NOTES



W

- A Component is the basic building block of React Applications.

REACT INTERVIEW QUESTIONS

► .What is Vite?

► What is the Components?

- A Component is the basic building block of React Applications.
- Internally A React Component is a JavaScript function.
- And a React elements is a JavaScript Object

► What is the difference between Class Components and Functional Components ?

• Functional Components :

- Functional Components are the JavaScript functions that return JSX(HTML like syntax).
- Earlier before the introduction of Hooks, Functional Components could not store state and they Could not use lifecycle methods.
- But when React Hooks Came in (React 16.8) Functional Components can use Hooks to Store state and can Handle lifecycle and side effects. Functional Components are preferred in modern React applications

• Class Components :

- Class Components are the legacy way to write React, heavily relying on Object-Oriented principles.
- Before React Hooks existed, Only class components could store State and Lifecycle methods. Earlier Functional Components were stateless.
- Use this.state and lifecycle methods, More boilerplate code and this keyword usage. Mostly found in legacy applications

► Can we pass one component to another component? Write a code example.

- Internally a React component is just JavaScript function and a React element is like JavaScript Object. That means you can pass them into other functions just like any other variable. In React terminology, this is called **Composition**.
- There are two standard ways to do this :
- **Using children Prop (The Wrapper Pattern)** : This is most common way. Every React component automatically receives a special prop called children. It represents whatever you put between the opening and closing tags of a component. This is perfect for creating reusable wrappers, like Modals, Layouts, or Cards.
- **Passing Component as named Props (The Slot Pattern)**: Sometimes children isn't enough because you need to inject components into very specific places (like a Left Sidebar and a Main Content area). In this case, you can pass components exactly like standard variables.

► What is a Higher-Order Component (HOC)?

- HOC is a function that takes a component as an argument and returns a new enhanced component. HOC does NOT modify original component.
- **First Understand the Problem HOC Solves** : Imagine you have multiple components: User Page, Product Page, Order Page. And all of them need own loading logic, authentication check, logging, API error handling. So without HOC we need to write the same duplicate code everywhere. So using HOC we can reuse the code logic.
- It is an implementation of the decorator pattern, which is used to reuse component logic (like authentication, data fetching, or error handling).
- **Do we still use HOC?**
 - HOCs were heavily used in older React codebases. Libraries like Redux use them extensively with functions like `connect()`
 - However, in modern React, HOCs are replaced by **Hooks & Custom Hooks**.
 - These are the standard way to share logic, because Hooks create less nested "wrapper hell" in the component tree.

► Give an example of an HOC :

- One of the most common examples is: Authentication HOC which is Used to protect pages/routes.
- **Redux (connect)** : This is the most famous HOC in React history. Before Hooks existed, if a component needed to read global state from a Redux store, you had to wrap it like this: `export default connect(mapStateToProps)(MyComponent);`.

- **React Router (withRouter):** In older versions of React Router, if a component was buried deep in the UI and needed access to the URL parameters or navigation history, you wrapped it in `withRouter(MyComponent)` to inject that data into its props.
- **Material-UI (withStyles):** Popular component libraries used HOCs to inject CSS-in-JS styling into standard components.

► Have you used HOCs anywhere in your application

- In modern React applications, Hooks are preferred more than HOCs for reusable logic.
- But I understand HOCs well because many legacy React applications still use them for reusable logic like authentication and loading handling.
- For example, I created a `withAuth` HOC to protect dashboard pages. Instead of checking login conditions inside every component, I wrapped the components using the HOC so the authentication logic stayed reusable and centralized.
- I have also seen HOCs used in libraries like React Redux where components are wrapped using functions like `connect()`.
-

► What is a Hook in React?

- Hooks are special built-in React functions that allows Functional Components to use React's internal features —like state, lifecycle methods, context, and refs.
- Hooks were introduced in React 16.8. Earlier before the introduction of Hooks, Functional Components could not store state and they Could not use lifecycle methods. Only Class Components could use these. But when React Hooks Came in modern React Functional Components can use Hooke to
 - Store state into React's memory : using `useState()`
 - Handle lifecycle and side effects : using `useEffect()`
 - Share data globally : using `Prop Drilling?()`
 - Access DOM / store mutable value : using `useRef()`

► What is `useState` vs `useReducer`? When to use which?

- **`useState`** : `useState` is React's simplest Hook. It is perfect for managing state of simple, independent variables.

- **useReducer** : useReducer is designed for complex state logic where the next state depends on the previous state, or multiple variables must change together in perfectly synchronized lockstep.
- **When to use which?** : Neither is inherently faster. useReducer and useState are both state management Hooks. useState is better for simple state, while useReducer is better for complex state transitions and centralized update logic.

► What is the useRef hook? When should it be used? What is forwardRef and useImperativeHandle?

• useRef (The Silent Memory & The Pointer)

In React, useState is loud: every time it changes, it shouts at the component to re-render. useRef is silent memory. It holds data, but when the data changes, the component does **not** re-render.

It has two distinct use cases:

Use Case A: Pointing to a Physical DOM Element Sometimes you need to bypass React's virtual state and touch the physical HTML element (e.g., to force the browser to focus an input, or to read the physical height of a div).

Use Case B: Storing Mutable Data (Without Re-rendering) Imagine you want to build a stopwatch. If you store the setInterval ID in useState, updating it might trigger an infinite rendering loop. You store it in a ref because it's purely behind-the-scenes memory.

- Java Analogy: A private class variable that is used for internal calculations but never serialized into the JSON response.

2. forwardRef (Passing the Pointer)

The Problem: By default, custom React components **do not** accept a ref prop. If you create a <MyCustomInput /> and try to pass a ref to it from a parent, React will throw a warning and ignore it. It protects components from having their internals messed with from the outside.

The Solution: forwardRef is a wrapper function that tells React, "It is okay, I am explicitly allowing the parent to pass a pointer down into my internal HTML."

useImperativeHandle (The Facade Pattern)

This is a senior-level concept.

If you use `forwardRef`, you are handing the parent component the **entire raw DOM node**. The parent can do whatever it wants: change the styling, delete the node, mess with the values. This breaks encapsulation.

- **Java Analogy:** It is like making your database connection object public. Anyone can accidentally call `.dropDatabase()`. Instead, you want to use the **Facade Pattern**: you give the parent an Interface that only exposes specific, safe methods (like `.connect()` and `.disconnect()`).

`useImperativeHandle` is React's Facade pattern. It intercepts the ref and allows you to define exactly what methods the parent is allowed to call.



•

► **What are the rules of hooks? Why do Hooks have rules? What happens if we violate them?**

- **Why Do Hooks Have Rules?**

- Cause React internally tracks Hooks based on the **order in which they are called**. On every re-render, React expects the Hooks to be called in the exact same order. If the order changes, React gets confused about which state belongs to which Hook.

- **Rules of hooks?**

- Rule #1: Only Call Hooks at the Top Level
- Rule #2: Only Call Hooks Inside React Components or Custom Hooks
- Rule #3: Custom Hooks Must Start With "use"
- Rule #4: Keep Hook Order Consistent

► **What is a Custom Hook? How do you define Custom Hook?**

- A **Custom Hook** is simply a JavaScript function that extracts component logic so it can be reused across multiple components.
- **Why Custom Hooks Exist?** Imagine: Many components need:
 - API calling logic
 - loading logic

- form logic
- authentication
- Instead of duplicating code, we create reusable Hook logic.
- **Custom Hook** is defined exactly like a normal JavaScript function, but it **must** follow two strict rules:
 - The function name **must start with "use"** so that React can internally identifies Hooks using this naming convention. (e.g., useFetch, useAuth).
 - It must call at least one other React Hook inside it.

► What Custom Hooks have you created in your application?

- In React applications, I prefer using Custom Hooks for reusable business logic instead of duplicating Hook logic inside components. For example, I created:
 - useFetch() for centralized API calling and loading handling
 - useAuth() for authentication state management. To check if a user's JWT token is still valid and globally expose their user profile data to the UI
 - useForm for managing form inputs and validation
 - useDebounce for optimized search input handling. If you have a search bar that calls your backend, you don't want to hit the API on every single keystroke. A debounce hook waits until the user stops typing for 500ms before triggering the API call.

► What is the difference between a Custom Hook and a normal JavaScript function?

- Hooks are nothing but Javascript Functions only, BUT there's a key difference.

Normal JS Function	Custom Hook
No Hook rules, it Can be named anything (e.g., calculateTotal, formatDate).	Follows Hook rules, like it must start with the word use (e.g., useWindowSize).
Cannot call React Hooks. React will throw a fatal error.	Can call other React Hooks like useState/ useEffect.

Has no concept of the React UI lifecycle.	Tied to the React component lifecycle (it knows when the component mounts or unmounts).
It is mainly used for general reusable logic (math, formatting strings, parsing JSON).	It is mainly used for Stateful UI logic (fetching data, tracking mouse position, managing forms).

► What are Higher-Order Components (HOCs) vs Custom Hooks vs Render Props? Give examples.

- HOC is a function that takes a component as an argument and returns a new enhanced component. But the Problems With HOCs are as applications grow: This becomes: Wrapper Hell.

```

withAuth(
  withLogger(
    withTheme(
      Component
    )
  )
)

```

- **Render Props** : A component shares logic by passing data through a function prop. Before Hooks became popular, Render Props were extremely common. But Problems With Render Props is Deep nesting which becomes: Callback Hell
- **Custom Hooks (Modern React)** : A Custom Hook is simply a JavaScript function that extracts component logic so it can be reused across multiple components. Instead of duplicating code, we create reusable Hook logic. Commonly used for useFetch(), useAuth(), useForm(), useDebounce(), usePagination(). Compared to HOCs and Render Props, Custom Hooks are they provide cleaner code, better reusability, and avoid the nesting problems associated with HOCs and Render Props.

•

•

► **If you have Redux Store and Local Storage, which one would you choose for security purposes and why?**

- For security purposes, **neither** Redux Store nor Local Storage is considered truly secure for sensitive data, because both store the token on the client side. However, if forced to choose between the two,
- **Redux Store** : Redux Store is safer than local storage. This is because Redux stores data in a temporary in-memory state. It gets clear on refresh
- **Local Storage** : Local Storage is Persistent browser storage which is stored in browser's disk. It does not get clear on refresh/browser restart. This is why it is highly vulnerable to Cross-Site Scripting (XSS) attacks.
- Best approach to secure highly sensitive data like JWT tokens—I would instruct the backend to send the JWT in an **HTTP-Only secure Cookie**.
- An HTTP-Only Cookie is an instruction to the browser to Store the token(JWT),and send it back to the server with every API request. Since JavaScript cannot access HttpOnly cookies, This is the most secure in modern approach and prevents XSS attacks aimed at stealing tokens.

► **What is Context API? What problem does Context API solve?**

- Context API is a React feature used to share data globally between components without manually passing props (without prop drilling) at every level.
- The Context API primarily solves the problem of **Prop Drilling** (also known as threading props).
- **What is Prop Drilling?** In standard React, data is passed top-down from parent to child via props. If a deeply nested child component needs data from a high-level parent, you have to pass that data through every single intermediate component—even if those middle components don't care about the data at all.
- Context API creates a global 'channel' that allows any component to access that data directly, bypassing the middleman.

► How do you implement Context API?

- Implementing Context always takes three exact steps:
 - **Create** : First, you initialize the Context. You usually do this in its own file so it can be exported and imported anywhere.
 - **Provide** : Next, you wrap the top-level part of your application with a Provider. Anything placed inside this wrapper will have access to the data.
 - **Consume** (Using the useContext Hook) : Finally, you go to the exact component that needs the data, completely ignoring where it lives in the tree. We use the useContext hook to pull the data directly out of the ether.

► What is Redux? Other than Redux, what state management solutions can be used?

- Redux is A centralized state management library for JavaScript applications. Instead of storing data separately in many components, Redux stores application data in One Central Store
- Mainly used with: React, Angular, Vue. but Redux itself is independent of React.
- **The 3 Core Building Blocks of Redux :**
 - **1. The Store (The Database)** : This is the single source of truth. It is literally just a massive JavaScript object holding your global data (e.g., the current user, shopping cart items, theme settings). Java Analogy: A global ConcurrentHashMap or an embedded Redis cache.
 - **2. Actions (The Events / DTOs)** : Components cannot change the Store directly. If a user clicks an "Add to Cart" button, the component must create an Action. An Action is a simple object containing a type (what happened) and a payload (the data). Java Analogy: An Event payload sent to a message broker (like Kafka or RabbitMQ) or a Data Transfer Object (DTO).

JavaScript

// An example of an Action

```
const addItemAction = {  
  type: 'CART/ADD_ITEM',  
  payload: { id: 101, name: 'Wireless Mouse' }  
};
```

- **3. Reducers (The Service Layer)** : A Reducer is a function that actually updates the Store. It listens for Actions. When an Action is dispatched, the Reducer takes the current state of the Store, looks at the Action, applies the business logic, and returns a brand new **updated State**. **Java Analogy**: The @Service class that processes the DTO and saves the new entity state.

JavaScript

// An example of a Reducer

```
function cartReducer(currentState = [], action) {  
  if (action.type === 'CART/ADD_ITEM') {  
    // Returns a NEW array with the existing items plus the new payload  
    return [...currentState, action.payload];  
  }  
  return currentState;  
}
```

The Redux Data Flow (Step-by-Step)

1. **Component**: The user clicks a button in the UI.
2. **Dispatch**: The component "dispatches" an Action ({ type: 'LOGIN', payload: userData }).
3. **Reducer**: The Store catches the Action and hands it to the Reducer. The Reducer calculates the new state.
4. **Store Updated** : The Store updates itself with the new state.
5. **UI Update**: Any component that is "subscribed" to the Store is automatically re-rendered with the fresh data.

► **What is Redux Toolkit? What state management alternatives exist (Zustand, Recoil, MobX)?**

► **Redux Toolkit (Modern Redux) :**

- In older React codebases, Redux was notorious. It requires a lot of boilerplate code just to create a simple counter, you needed: actions/ reducers/constants/ types/ store/)

- Today, the industry uses **Redux Toolkit (RTK)**. Redux Toolkit is the official Redux package that simplifies Redux development by reducing boilerplate code and providing built-in tools for state management, asynchronous operations, and store configuration.
- Main Features of Redux Toolkit :
 - **configureStore()** : Automatically sets up the Redux store with the best security practices, connects to the browser's Redux DevTools, and automatically configures Redux Thunk for API calls so you don't have to.
 - **createSlice()** : This is the game-changer. Instead of writing Actions and Reducers separately, you write a "Slice". RTK automatically generates the Action objects and the Action Types for you based on the Reducer names.
 - **Built-In Immer.js** : Remember how Reducers must be strictly immutable? In legacy Redux, copying deep objects was a nightmare. RTK includes a library called Immer. Immer lets you write code that looks like you are mutating the state (e.g., `state.user.name = "Alice"`), but under the hood, it safely translates that into a perfectly immutable copy.
 - **createAsyncThunk()**

► Modern State Management Alternatives :

► 1. Zustand : A lightweight state management library. It uses a single, global store, but requires absolutely zero boilerplate, no "Providers" wrapping your app, and no complex reducers. It relies entirely on Custom Hooks. **Why use it?** It is incredibly fast, simple to read, and solves 95% of the problems Redux solves with 10% of the code.

► MobX (The Angular / OOP Approach) : It relies heavily on **Observables** (like RxJS) and Object-Oriented principles.

- **How it works:** Instead of functional, immutable data flows, you create literal JavaScript Classes, mark their properties as `@observable`, and mark their methods as `@action`. React components "listen" to the class and re-render when a property mutates.
- **Why use it?** If a team is migrating from Angular to React, or heavily prefers OOP patterns over Functional Programming, MobX is the go-to. However, it is losing market share to Zustand.

► Recoil / Jotai (The Atomic Approach)

Redux and Zustand are "Monolithic" —they use one giant object to store everything. Recoil and Jotai use an "Atomic" model.

- **How it works:** You create tiny, independent fragments of state called "Atoms" (e.g., `const themeAtom = atom('dark')`). Components can subscribe to individual atoms anywhere in the tree.
- **Why use it?** It is highly specialized. You only use this if you are building an app with highly interconnected, deeply complex UI nodes (like a web-based spreadsheet, a canvas drawing tool like Figma, or a complex node-graph visualizer).



▶

▶

▶

▶ **What is a Reducer? How do you implement a Reducer? Write a code snippet. What is the Reducer lifecycle? How does Reducer work internally?**

- **Reducer** : A Reducer is a **pure JavaScript function** that takes the current state and an action object as arguments, and returns a completely new updated state object.
- Syntax : (state, action) => newState
- It is called a "reducer" because it takes multiple values over time (actions) and accumulates them into a single value (the state).
- **Reducer Lifecycle and Internal working ?**

How does this actually work under the hood? The term "Reducer" comes from the JavaScript array method `Array.prototype.reduce()`. Just like an array reduce function iterates over a list of items and accumulates a single value, a Redux Reducer evaluates a stream of actions over time to accumulate the current state of your UI.

Here is the exact lifecycle loop:

1. **Trigger (Dispatch)**: A user clicks "Add to Cart". The React component fires an action: `dispatch({ type: 'ADD_TO_CART', payload: 1 })`.
2. **Interception**: The Redux Store catches this action.
3. **Execution (The Reducer)**: The Store calls your `appReducer(currentState, action)`.
4. **Calculation**: The switch statement finds 'ADD_TO_CART'. It creates a new JavaScript object in memory, updates the `cartItems`, and returns it.
5. **State Replacement**: The Redux Store deletes the old state and replaces it with the new object.
6. **Broadcast**: The Store notifies React that the state has changed. React efficiently re-renders only the components (like the Cart Icon in your header) that care about `cartItems`.

▶ **What is Redux Thunk? What problem does Thunk solve?**

- **Problem with Redux** : Redux Reducers are Pure synchronous function. It takes an Action, calculates the new state, and returns it immediately.

- It means: Redux by default Cannot handle async operations like API calls, login API, fetching users, delayed dispatch.
- So this is **WRONG** inside reducer: `function reducer(state, action) { fetch("/users"); //cant do API calls return state; }`
- So to solve this Redux Thunk comes in which is a middleware that sits between the moment you dispatch an Action and the moment it reaches the Reducer. Basically it allows Redux to handle asynchronous operations (API calls, delayed actions, async business logic).
- It's worth noting that in modern React development, we use **Redux Toolkit (RTK)**. RTK actually comes with Thunk built-in automatically, and provides a utility called `createAsyncThunk` which removes almost all the boilerplate code needed to write these API calls."

► **Difference Between Normal Redux and Thunk**

•

Normal Redux	Redux Thunk
Only sync actions	Async actions supported
Action = object	Action = function possible
No API handling	API handling supported

•

- **How can you optimize a React application?// How can we increase the performance of a React application?// What techniques have you used to optimize React application performance?**
- React performance issues mainly happen because of: Unnecessary Re-rendering.
 - We can optimize the application by: **preventing unnecessary re-renders-**
 - **React.memo()** : **It Caches the Component**, preventing unnecessary component re-renders. We can wrap child component using **React.memo()**. Now component only re-renders if props actually change.

- **useMemo()** : It caches heavy calculations, preventing heavy calculations like filtering, sorting to execute on every render. we can use **useMemo()** to caches the result of previous calculation and only re-calculates it if the raw data changes.
 - **useCallback()** : It caches Functions, preventing unnecessary function recreation and unnecessary child renders. Using **useCallback()** every render creates a new function reference. Function reference stays stable.
 - **Code splitting and lazy loading components** : using Lazy loading (**React.lazy**) & **Suspense** we can dynamically Load components only when the user navigates to them.
 - **optimizing API calls** : Mostly in search field we can enable debouncing which will wait few time before API call instead of calling api on every keystroke.
- **What is the difference between useEffect and useLayoutEffect? What does the dependency array do? How do you handle side effects and cleanup?**
- **A side effect means:** Anything outside of rendering UI. Examples: API calls, Event listeners, Timers, Local Storage access, DOM manipulation, WebSocket connections.
- **useEffect (Asynchronous - Use this 99% of the time)** : useEffect is a Hook used to perform side effects in a React component. useEffect runs after after React has updated the DOM the browser has painted the screen. It runs in the background. Because it doesn't block the browser from painting, your app feels fast and responsive.

Render UI



Paint UI on screen



Run useEffect

- **useLayoutEffect (Synchronous - Use this 1% of the time)** : useLayoutEffect is very similar to useEffect. The difference is it runs after React updates the DOM, but before the browser paints the screen to the user. It physically blocks the browser from drawing the UI until your code finishes executing.

Render UI



Run `useLayoutEffect`



Paint UI on screen

► `useLayoutEffect` Use for: Measuring DOM size, Scrolling calculations, Animations, Layout adjustments

► What does the Dependency Array do?

- Every time a component's state changes, the component re-renders (the function runs from top to bottom). Dependency array controls: WHEN the effect should run. The **Dependency Array** is how you tell React to skip running your `useEffect` logic on certain renders.

► Syntax :

```
useEffect(() => { }, [dependencies]);
```

► **Case 1: No Dependency Array : (The Danger Zone):** The effect runs after every single render. You almost never want this. If you put an API call here, and that API call updates the state, the state change triggers a re-render, which triggers the API call again... resulting in an infinite loop.



```
useEffect(() => {
```

```
  console.log("Running");
```

```
});
```

► **Case 2: Empty Dependency Array [] : (The @PostConstruct):** The effect runs exactly once when the component first appears on the screen (mounts). This is where you put your initial API fetches.

```
useEffect(() => {  
  
  console.log("Mounted");  
  
}, []);
```

► **Case 3: Array with Variables [id, user]:** The effect runs on the first mount, and then only re-runs if the value of search changes.

```
useEffect(() => {  
  
  console.log("Search Changed");  
  
}, [search]);
```

► **How do you handle Side Effects?** Side effects belong inside: `useEffect()`

When a component is removed from the screen (unmounts), React needs to clean up those side effects so they don't keep running in the background and cause memory leaks. Cleanup prevents memory leaks by removing timers, event listeners, subscriptions, and other resources when a component unmounts or dependencies change.

How to handle cleanup: You simply return a function from inside your `useEffect`. Think of this returned function exactly like a `@PreDestroy` method in Spring or a `finally` block.

```

useEffect(() => {
  // 1. The Setup (The Side Effect)
  console.log("Connecting to WebSocket...");
  const connection = new WebSocket('ws://localhost:8080/chat');

  // 2. The Cleanup (The Teardown)
  return () => {
    console.log("Component unmounting. Closing connection...");
    connection.close();
  };
}, []); // Empty array means connect on mount, disconnect on unmount

```



•

- What are lifecycle methods vs hooks equivalents?
- Lifecycle Means A React component that goes through different phases like :

Mount (Component created)



Update (Component re-renders)



Unmount (Component destroyed)

React provides lifecycle methods/hooks to execute code during these phases.

In older React Class Components, lifecycle methods were based on **time**: When does this component load? When does it update?

In modern Functional Components, Hooks (specifically `useEffect`) are based on **synchronization**: What data are we watching, and what should we synchronize when that data changes

In modern React, `useEffect` is used to replace `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`. `React.memo` is used similarly to

shouldComponentUpdate, while Error Boundaries handle functionality similar to componentDidCatch.

Class Lifecycle	Hook Equivalent
componentDidMount	useEffect(() => {}, [])
componentDidUpdate	useEffect(() => {}, [dependency])
componentWillUnmount	return cleanup function
shouldComponentUpdate	React.memo
getDerivedStateFromProps	Usually avoid / useEffect
componentDidCatch	Error Boundary

► What is Debouncing? When do you need it and why? Where else (besides search) can it be used?

- **Props (Properties)** : Debouncing is a performance optimisation technique that delays the execution of a function until the user stops performing an action (Typing, Scrolling) for a specified amount of time.
- We Need Debouncing Because some operations like API request, Database queries etc are expensive.
- Without debouncing: Every keystroke will call -> Expensive operation which can Slow UI, Increase server load, Waste bandwidth, Create unnecessary processing
- **Beside**
- **Form Validation** : like Email Validation
- **Username Availability Check** : in Registration form Check: Does username exist?
- **Auto-Save Forms** :
- **Calculations like EMI Calculator/ Tax Calculator** where User continuously changes values. Instead of: Recalculate every keystroke we can use debouncing.
- **Analytics Tracking** : Track User Activity. Without debounce: Thousands of events. With debounce: Meaningful event collection
-

► What is the difference between state and props? What is "Lifting State Up"?

- **Props (Properties)** : Props are nothing but data passed from Parent Component to Child Component. Props are generally : Read-only value and a child component should not modify its own props. Props data flow one way : Parent → Child
- **State** : State is internal data managed inside a component that can change over time. State is mutable via setter **function(setCount)**. It is used for: Counters, Forms, Search text, API responses, Toggle buttons.
- **What happens when state changes?** When state changes, React re-renders the component and updates the UI accordingly.
- **What is Lifting State Up?** Lifting State Up is a fundamental React architectural pattern. It is the process of moving state from a child component to the nearest common parent component so that multiple components can share and synchronize the same state.

► What is the difference between Controlled and Uncontrolled components? How do you handle forms in React?

- In traditional HTML, form elements (like `<input>`, `<textarea>`, and `<select>`) naturally maintain their own internal state. If a user types into an input field, the browser just remembers it.
- In React, we usually don't want the browser managing data; we want React's memory (useState) to manage everything so we can strictly control it. This brings us to the two ways to handle forms
- **Controlled Component** : A Controlled Component is a form element whose value is controlled by React state. Meaning: React State = Single Source of Truth. Every time the user types a single character, the input triggers a React state update. React then re-renders the input with the new state. The DOM never actually remembers what the user typed; it only displays whatever React tells it to display.
- **Uncontrolled Component** : An Uncontrolled Component acts like standard HTML, it stores its state inside the DOM. Every time the user types a single character, the input Value is stored in: HTML Input Element and not in React State.

► What is Reconciliation and how does React's diffing algorithm work?

- To understand Reconciliation, we first have to understand the problem React was trying to solve: The actual browser DOM is incredibly slow. If you have a table with 1,000 rows and you change one piece of text in one row, traditionally, JavaScript might force the browser to recalculate the layout and repaint a massive chunk of

that table. It is the frontend equivalent of dropping an entire database table and re-inserting all 1,000 records just because you updated one user's email address.

- Here is how React solves this using the Virtual DOM, Reconciliation, and its Diffing Algorithm.
- **The Virtual DOM (The In-Memory Cache)** : The physical browser DOM (Document Object Model) is essentially an API provided by the browser to manipulate HTML. It is slow. Every time you change a node, the browser has to recalculate CSS styles, figure out layouts, and repaint pixels on the screen.
- React rarely touches the actual HTML DOM directly. Instead, it creates a Virtual DOM.
- Virtual DOM is a lightweight in-memory representation of the real DOM. React uses it to compare UI changes efficiently and update only the necessary parts of the real DOM.
- **Reconciliation** : Reconciliation is the overarching process React uses to sync the Virtual DOM with the Real DOM. It determines what changed in the UI and what needs to be updated in the real DOM. When a user clicks a button and you update a State variable (e.g., using `useState`), here is the exact pipeline:
 1. React builds a brand **New Virtual DOM** tree representing how the screen should look right now.
 2. React compares the **New Virtual DOM** against the **Old Virtual DOM** (from before the button click).
 3. React figures out the exact difference between the two trees.
 4. React sends a minimal "patch" instruction to the Real DOM, updating only the specific pixels that changed.
-
- **The Diffing Algorithm (The Engine)** : The Diffing Algorithm (The Engine)

Step 3 in the process above is where the magic happens. Comparing two tree structures to find the differences is a classic computer science problem. The standard algorithm to do this has a time complexity of $O(n^3)$. For a UI with 1,000 elements, that would require 1 billion comparisons. That would freeze the browser.

React implements a heuristic **Diffing Algorithm** that achieves this in $O(n)$ time by making two strict assumptions:

Rule 1: Different Element Types = Total Rebuild

If React is comparing a node in the Old Tree to the New Tree, and the HTML tag or Component name has changed (e.g., a `<div>` turned into a `<span>`, or `<Header>` turned into `<Footer>`), React completely stops comparing. It assumes the entire branch is entirely new. It destroys the old DOM node, unmounts all children, and builds the new node from scratch.

Rule 2: Same Element Types = Efficient Update

If the element type is the exact same (e.g., an `<input>` is still an `<input>`), React keeps the physical DOM node alive. It simply looks at the attributes and updates only what changed.

- Old: `<input className="empty" disabled={false} />`
- New: `<input className="error" disabled={true} />`
- Action: React tells the browser: "Keep the input on the screen, just swap the class to 'error' and add the 'disabled' flag."

Rule 3: Recursing on Children (The key Prop)

This is the most common interview gotcha. Imagine you have a list of items: `<li>Apple</li>` and `<li>Banana</li>`. If you add `<li>Cherry</li>` to the **bottom** of the list, React easily compares them top-to-bottom, sees Cherry is new, and just appends it.

But what if you add `<li>Cherry</li>` to the **top** of the list? React compares the first item (Apple vs Cherry) and thinks it changed. It compares the second item (Banana vs Apple) and thinks it changed. It ends up inefficiently mutating every single item in the list!

To solve this, React requires you to provide a **key prop** when mapping over arrays. `<li key="101">Apple</li>` By giving items unique IDs (like primary keys in a database), React's diffing algorithm can instantly recognize: "Oh, Apple and Banana didn't change, they just shifted down. I will keep those DOM nodes intact and just insert Cherry at the top."

To help you visualize exactly how this $O(n)$ algorithm evaluates a tree and patches the DOM, here is an interactive visualization. Play specifically with the difference between inserting a child "With Keys" versus "Without Keys" to see how drastically it affects performance.

React Fiber (The Concurrency Engine) :

- **First lets understand the problem** : Before React 16 (released in 2017), React rendering was synchronous and single-threaded. If your application had a massive component tree and React started reconciling it, it could not be interrupted. It

blocked the main JavaScript thread completely until it finished. If the user tried to type in a text box or a CSS animation was trying to run while React was doing this heavy math, the application would freeze and stutter.

- React Fiber is The new React reconciliation engine introduced in React 16. **Instead of traversing the entire component tree in one giant, unstoppable sweep, Fiber allows React to:**

1. Pause work and come back to it later.
2. **Assign priority** to different types of work.
3. **Reuse previously completed work** or abort it if it's no longer needed.
4. React Fiber was introduced to support incremental rendering, prioritization, interruptible rendering, and better responsiveness for complex applications.

► What is the key prop in React? Why shouldn't you use array indices as keys?

- The key prop is a special React attribute used to uniquely identify elements in a list during reconciliation so it can efficiently determine which items were added, removed, or updated..
- Why is key important? It helps React: identify changed items, identify added items, identify removed items, optimize rendering.
- **Why Shouldn't You Use Array Index as Key?** : Array indices change when items are: inserted, deleted, reordered, filtered.
- This can cause: unnecessary re-renders, incorrect UI updates, state-related bugs

► Context API vs Redux :

- Both allow Global State Sharing. But the Difference is
 - Context API focuses on: Sharing Data. Context API is Good for Simple shared state like Authentication, Theme, Language, User Preferences etc.
 - Redux focuses on: Managing Complex State like Shopping Cart, Dashboard Data, Complex Workflows, Large Applications.

► What are React Fragments and Portals? When to use them?

- React Fragment : The Problem : React components must return Only one parent element. A Fragment groups multiple elements without creating an extra DOM node.

